

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Programación 2

Trabajo Práctico Integrador

Food Store – Sistema de Gestión de Pedidos de Comida

Alumno: Valentín Altube

Repositorio: github.com/valentin0987/Integrador_programacion_2

Video demostrativo: <https://youtu.be/N82E76JqZnl>

Año: 2026

Índice

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	4
3. Funcionalidades Implementadas	5
4. Dificultades y Soluciones	6
5. Bibliografía / Webgrafía	7

1. Marco Teórico

Java 21

Java es un lenguaje de programación orientado a objetos, multiplataforma y de tipado estático. Para este proyecto se utilizó Java 21, la versión LTS más reciente al momento del desarrollo, que incorpora mejoras en el manejo de switch expressions y otras características modernas del lenguaje.

Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el software en torno a objetos que combinan estado (atributos) y comportamiento (métodos). Los pilares aplicados en este proyecto son:

- Encapsulamiento: los atributos de las clases son privados y accesibles mediante getters y setters.
- Herencia: todas las entidades extienden de la clase abstracta Base (id, eliminado, createdAt).
- Polimorfismo: la interfaz Calculable define calcularTotal(), implementado por Pedido.
- Abstracción: la clase Base centraliza los atributos comunes de todas las entidades.

Colecciones en Java

Las colecciones son estructuras de datos dinámicas del paquete java.util. En este proyecto se utilizó ArrayList para almacenar en memoria las entidades durante la ejecución del programa, reemplazando la necesidad de una base de datos.

Manejo de Excepciones

Java permite crear excepciones propias extendiendo la clase Exception. Se implementaron tres excepciones personalizadas: EntidadNoEncontradaException, StockInvalidoException y DatoInvalidoException, que permiten manejar errores de dominio de forma específica y clara.

2. Decisiones Técnicas y Arquitectura

Estructura de paquetes

El proyecto se organizó en los siguientes paquetes:

- integrador_programacion_2: clase Main con el menú de consola.
- integrador_programacion_2.entities: clases del modelo de dominio.
- integrador_programacion_2.enums: enumeraciones Rol, Estado y FormaPago.
- integrador_programacion_2.exception: excepciones propias del dominio.
- integrador_programacion_2.service: servicios con la lógica de negocio y las colecciones.

Separación de capas

Se adoptó una arquitectura de tres capas:

- Capa de entidades: clases del UML con atributos, constructores, getters/setters y toString().
- Capa de servicios: lógica de negocio, validaciones y gestión de las colecciones en memoria.
- Capa de presentación: menú de consola con Scanner, validación de entradas y mensajes al usuario.

Soft Delete

Todas las bajas son lógicas: en lugar de eliminar el objeto de la colección, se marca eliminado = true heredado de Base. Esto preserva el historial de datos y evita inconsistencias en pedidos que referencian entidades eliminadas.

Interfaz Calculable

La clase Pedido implementa Calculable con calcularTotal(), que recorre la lista de DetallePedido y suma los subtotales para actualizar el total del pedido, garantizando consistencia entre los detalles y el total final.

3. Funcionalidades Implementadas

Gestión de Categorías

- Listar categorías activas (no eliminadas).
- Crear categoría con validación de nombre no vacío y unicidad.
- Editar nombre y descripción por ID.
- Eliminar lógicamente con confirmación S/N.

Gestión de Productos

- Listar productos activos con categoría asociada.
- Crear producto con validaciones: precio ≥ 0 , stock ≥ 0 , categoría existente.
- Editar todos los campos del producto.
- Eliminar lógicamente con confirmación.

Gestión de Usuarios

- Listar usuarios activos con ID, nombre, apellido, mail y rol.
- Crear usuario con validación de mail único y no vacío.
- Editar datos con revalidación de mail en caso de cambio.
- Eliminar lógicamente manteniendo historial de pedidos.

Gestión de Pedidos

- Listar pedidos con detalles (productos, cantidades, subtotales).
- Crear pedido seleccionando usuario y agregando N detalles usando `addDetallePedido()`.
- Calcular total automáticamente con `calcularTotal()` de la interfaz `Calculable`.
- Actualizar estado (enum `Estado`) y forma de pago (enum `FormaPago`).
- Eliminar lógicamente con confirmación.

4. Dificultades y Soluciones

Organización de paquetes en NetBeans

Al iniciar el proyecto, todas las clases estaban en el paquete raíz. Se reorganizaron en subpaquetes usando Refactor → Move de NetBeans, que actualizó los imports automáticamente.

Lectura de entradas con Scanner

Se usó exclusivamente `nextLine()` para todas las entradas y se parsearon los valores numéricos con `Integer.parseInt()`, `Double.parseDouble()` y `Long.parseLong()` dentro de bloques try-catch, garantizando que entradas inválidas no rompan el flujo del programa.

Clase duplicada en Main.java

Al pegar el código del menú, quedaron dos declaraciones de clase en el mismo archivo. Se resolvió eliminando el bloque duplicado, dejando una única clase con todo el contenido correcto.

Integridad referencial sin base de datos

Al eliminar categorías o productos referenciados en pedidos, se optó por el soft delete para no romper las referencias existentes. Los detalles de pedidos mantienen su referencia al objeto aunque esté marcado como eliminado.

5. Bibliografía / Webgrafía

- Documentación oficial de Java 21: <https://docs.oracle.com/en/java/javase/21/>
- Java Collections Framework:
<https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
- Repositorio del proyecto:
https://github.com/valentin0987/Integrador_programacion_2
- Video demostrativo: <https://youtu.be/N82E76JqZnI>
- NetBeans IDE: <https://netbeans.apache.org/>
- Consigna TPI – Programación 2, UTN 2026.